Video Demo Script & Presentation Deck

This document contains the exact slide deck structure, on-screen actions, and word-for-word spoken script for your 10-minute unlisted YouTube video submission.

Part 1: Customer Pitch & Portal Demonstration (Max 5 Minutes)

Target Audience: Head of Day-Ahead Trading & Power Procurement Desk at a UK Energy Utility.
Objective: Impress the client with business domain understanding, decision-making clarity, and risk mitigation tools.

Slide / Screen Sequence & Script

[00:00 - 00:45] The Day-Ahead Procurement Problem

- **On Screen**: Introduction Slide & Application Header
- **Spoken Script**:

> *"Hello everyone. Today, I am proud to present the pilot output of our UK Wind Day-Ahead Procurement & Risk Intelligence Platform, built specifically for your wholesale power purchasing desk.
>
> As an energy utility serving millions of consumers across Great Britain, your trading desk faces a critical daily challenge: at 11:00 or 12:00 GMT on Day D-1, gate closure strikes for the EPEX SPOT Day-Ahead auction. You must commit orders to buy thermal power from CCGT generators and peakers to cover your customer load. Every megawatt of wind power generated lowers your thermal purchasing cost.
>
> But wind is intermittent. If you over-rely on a wind forecast that fails to materialize, you are forced into real-time imbalance cash-out at punitive System Buy Prices. Conversely, if wind is under-forecast, you over-buy thermal power and spill surplus energy at a steep loss.
>
> Our platform solves this by delivering actionable visibility into the latest Elexon forecasts available at the exact second of gate closure, paired with quantitative error buffers to protect your trading margins."*

[00:45 - 02:15] Purpose 1 Demo: Day-Ahead Procurement Decision Workspace

- **On Screen**: Screen share of the web app on `Day-Ahead Decision` tab. Select Date `2026-01-15` and Gate Closure `12:00 GMT`.
- **Spoken Script**:

> *"Let's look at the live platform in action.
>
> Here on the Day-Ahead Decision dashboard, a trader begins by selecting their target delivery date—for instance, January 15, 2026—and their auction gate closure cutoff, such as the standard 12:00 midday or 11:00 EPEX Spot auction.

> Notice what the system immediately delivers: First, the Top Metric Bar shows our key operational parameters: Expected Wind Generation for the day is 283,000 MWh, with a forecasted peak of 14,200 MW and a minimum base floor of 8,500 MW. This minimum floor gives the desk confidence in the firm renewable baseload they can count on.
>
> Second, on the main interactive profile chart, the cyan line depicts the official National Grid WINDFOR forecast published right before our 12:00 gate closure. When delivery occurs, the green dashed line displays the actual outturn from FUELHH half-hour by half-hour.
>
> Third, click the 'Forecast Waterfall' button. This is a game-changer for your desk. It overlays the previous publication cycles—from 06:00, 09:00, through 15:00 and 18:00. This waterfall shows trajectory convergence: you can immediately see if weather models were steadily ramping wind expectations up or down leading into the auction, giving your traders directional conviction.
>
> Finally, you can export the full 48 settlement periods schedule directly to CSV for integration into your trade capture and ETRM systems."*

[02:15 - 04:00] Purpose 2 Demo: Quantitative Accuracy & Risk Buffering

- **On Screen**: Click over to the `Accuracy & Risk Analytics` tab.
- **Spoken Script**:

> *"Now let's turn to Purpose 2: How do your traders quantify error and protect their book?
>
> In the Accuracy & Risk tab, we analyze all January 2026 data. Across the entire month at 12:00 D-1 cutoff, the national wind forecast exhibited a Mean Absolute Error (MAE) of approximately 2,000 MW, with a Weighted Absolute Percentage Error (WAPE) of 17.3%, and an R-squared correlation of 0.77.
>
> Most critically, examine the Mean Forecast Bias metric: it shows +1,805 MW. This is a vital commercial finding: the day-ahead wind forecast had a systematic tendency to over-forecast generation in January 2026. If your desk bought power naively according to the raw forecast, you would have been systematically short in real-time, suffering cashout penalties!
>
> Look at the charts below:
>
> 1. The Daily Error Trend highlights exactly which days had calm atmospheric blocks versus volatile storm passages where errors spiked.
> 2. The Diurnal Error Profile breaks error down by hour of the day. You can see that errors are highest during the afternoon ramp (14:00 to 18:00)—which coincides exactly with the UK evening peak power price spike!
> 3. Under Wind Generation Regimes, we stratify accuracy: in High Wind conditions (>12 GW), absolute error is higher, whereas percentage error in Low Wind (<5 GW) presents the greatest relative volume risk.
> 4. To operationalize this, our P90 Reserve Buffer Calculator recommends an automated procurement buffer (+1,500 to +2,000 MW) that traders should contract in the day-ahead auction to ensure 90% statistical protection against short positions."*

[04:00 - 05:00] Summary & Commercial ROI

- **On Screen**: Summary ROI slide or Return to Overview cards.
- **Spoken Script**:

> *"To summarize the business value of this pilot:
>
> - Your desk gains sub-second clarity on the latest renewable generation forecasts matched to your specific auction cutoffs.
> - You replace guesswork with rigorous, quantitative error metrics.
> - You mitigate costly imbalance penalties through dynamic risk buffering.
>
> This platform is ready to deploy directly into your trading operations. Thank you, and I look forward to your questions."*

Part 2: Technical Lead Architecture & Codebase Review (Max 5 Minutes)

Target Audience: Internal Technical Lead / Principal Architect reviewing the codebase.
Objective: Clear knowledge transfer, architectural explanation, algorithm verification, and software engineering practices.

Slide / Screen Sequence & Script

[05:00 - 06:15] System Architecture & Data Pipeline

- **On Screen**: Architecture Diagram or Codebase Directory Structure (`backend/main.py`, `frontend/src/App.jsx`)
- **Spoken Script**:

> *"Welcome to Part 2 of the presentation. Here I'll walk through the technical architecture, data pipeline, design decisions, and code implementation.
>
> The system is architected as a lightweight, high-performance, containerized full-stack application:
>
> - Frontend: Modern React 18 built with Vite, styled with Tailwind CSS, using Recharts for responsive SVG data visualizations and Lucide React for UI instrumentation.
> - Backend: FastAPI in Python 3.11, providing asynchronous REST endpoints with automated OpenAPI docs, high-speed in-memory vector indexing, and static asset serving.
> - Data Source: Live Elexon BMRS API endpoints:
>   1. FUELHH: Half-hourly settlement outturn data filtered on fuelType='WIND'.
>   2. WINDFOR: Continuous multi-horizon forecast stream containing publishTime, startTime, and forecast generation in MW.
>
> We pre-ingested and bundled the full January 2026 dataset (1,488 half-hourly actual records and 19,856 forecast records) in `backend/data/` so the application runs deterministically with instant startup, zero cold-start latency, and full offline resilience."*

[06:15 - 07:45] Core Algorithm: Day-Ahead Matching Logic

- **On Screen**: Show `get_day_ahead_matches` and `get_forecasts` functions in `backend/main.py`.

- **Spoken Script**:

> *"Let's examine the core domain matching algorithm in `backend/main.py`.
>
> The key technical subtlety in UK power trading is temporal causality: For any delivery date D, trades must be placed on Day D-1 prior to gate closure time T (e.g., 12:00:00Z).
>
> In our `get_day_ahead_matches` function:
>
> 1. We compute `cutoff = (delivery_date - 1 day) at gate_closure_hour:00:00Z`.
> 2. For each hourly target time on delivery day D, we filter all WINDFOR records where `startTime == target_time` and `publishTime <= cutoff`.
> 3. We take `latest = max(candidates, key=lambda x: x['publishTime'])`. This strictly guarantees no future data leakage—the algorithm only sees what was physically published before the trader submitted their order.
> 4. We then pair this forecast with the actual outturn from FUELHH.
>
> Another domain discovery: WINDFOR provides hourly resolution forecasts, while FUELHH reports half-hourly settlement periods. In `get_forecasts`, we seamlessly map both half-hourly settlement periods (e.g. SP 23 & 24) to the corresponding hourly forecast window, computing the true half-hour average outturn for exact error computation."*

[07:45 - 08:45] Quantitative Error Engine

- **On Screen**: Show metrics computations in `backend/main.py` (`get_summary` and `get_metrics`).
- **Spoken Script**:

> *"The analytics engine computes all standard energy forecasting benchmarks:
>
> - MAE: Mean absolute error in MW.
> - RMSE: Root mean square error, penalizing extreme outlier spikes.
> - Mean Bias: Sum of (Forecast - Actual) / N. A positive bias signifies systematic over-prediction.
> - WAPE: Weighted Absolute Percentage Error, calculated as the sum of absolute errors divided by total actual generation, which avoids division-by-zero anomalies common with standard MAPE during low wind periods.
> - Correlation R^2: Calculated using sample covariance and variances.
> - Non-parametric Percentiles: Computed from P5 to P95 to construct empirical error distribution intervals without assuming a Gaussian distribution."*

[08:45 - 10:00] DevOps, Containerization & Testing

- **On Screen**: Show `Dockerfile`, `docker-compose.yml`, and terminal running `backend/test_app.py`.
- **Spoken Script**:

> *"Finally, let's cover development practices and deployment:
>
> 1. Containerization: We implemented a multi-stage Docker build in `Dockerfile`. The first stage builds the optimized React static assets using Node 20. The second stage packages the FastAPI Python environment and copies the compiled frontend into `frontend/dist`.

> FastAPI then mounts and serves both the REST API and the frontend from port 8000. This means `docker compose up --build` spins up the entire production system with a single command.
>
> 2. Cloud Readiness: We have included `Procfile` for Heroku/Render and `vercel.json` for Vercel, allowing immediate cloud deployment.
> 3. Automated Testing: In `backend/test_app.py`, we created an automated assertion test suite covering API health, summary calculation bounds, forecast timeline structures, and metric integrity. Running `python backend/test_app.py` completes in under 2 seconds.
>
> The codebase is clean, modular, and adhering to strict software engineering standards. Thank you for your review, and I welcome any technical questions."*